LangGraph & RAG

Building a Multi-Step Pipeline

Week 3 · Turing × UCSB × ACM

Irene Fan & Agnibha Sarkar | March 13, 2026

irenefanyiran@gmail.com · agnibha.sarkar@turing.com

Course Schedule

W1	Course Kickoff + Problem Scoping
W2	Agentic AI & Fundamentals
W3	**LangGraph & RAG ← YOU ARE HERE**
W4	HITL Systems
W5	Industry Realities of Building AI Products
W6	Architecture Design & Eng Design Doc
W7	Architecture Presentations + Critique
W8	Coding Copilots & Dev Cycle Integration
W9	End-to-End Implementation & Evaluation
W10	Demo Week

Today

1. Recap Week 2 — Quiz & Discussion Revisited
2. From Extraction to Classification
3. RAG: Why, Not Just How
4. LangGraph: Graphs Over Glue Code
5. Live Demo: CBRE Extract → Retrieve → Classify
6. Discussion
7. Assignment + Reading List

Quick Recap: Week 2

We built the extraction node. Today we connect it to the rest of the pipeline.

Agentic AI	Iterative workflows, not one-shot prompts
Four Design Patterns	Reflection, Tool Use, Planning, Multi-Agent
Reflection Deep Dive	Review loops that catch compound hazards
Evaluations	Measure systematically — the #1 predictor of success
Hands-On	Entity extraction from maintenance call transcripts

Today: connect Extraction → RAG Retrieval → Classification → Route

Let's Revisit: Food for Thought from Week 2

FT-2

"Reflection improves results, but every reflection step is another LLM call — more cost, more latency. When is reflection not worth it?"

Did anyone think about this during the week?

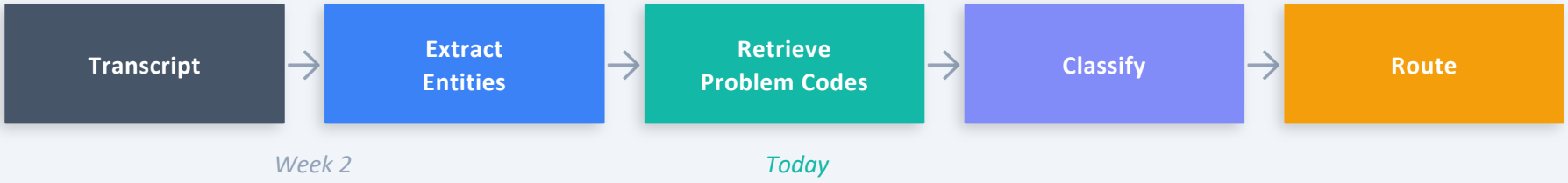
FT-5

"We used one LLM to generate and the same LLM to reflect. Is the model really "catching its own mistakes," or is it just getting a second chance with more context?"

Does it matter if the result improves either way?

From Extraction to Classification

Last week: Transcript → Entities. That's not enough to route the call.



To go from entities → route, we need:

1 Match

the extracted problem to a known problem code

2 Classify

using both the extracted entities AND the matched codes

The question is: how do we find the right problem codes?

The Naive Approach... and Why It Breaks

The Simple Way

```
"Here are all 21 problem codes.
Pick the best one."
```

This works. For 21 codes.

- Simple to implement
- Zero setup required
- Fine for 21 codes (~1,200 tokens)

But What If...

⚠ CBRE has 500 codes across regions?

⚠ Each code has a paragraph description + keywords?

⚠ You add historical work orders as context?

⚠ The prompt hits 100,000+ tokens — every call?

The prompt becomes a haystack. The LLM has to find the needle.

RAG: Retrieval-Augmented Generation

Search first. Pass only what's relevant. Generate from context.

1

Store

Your knowledge in a searchable format (vector store)

2

Retrieve

Only the relevant pieces for each query

3

Generate

A response using only the retrieved context

Why It Matters

	Without RAG	With RAG
Volume	All 500 codes in every prompt	Only 3-5 relevant codes per call
Cost	Scales with total knowledge size	Scales with relevance — stays low
Focus	LLM attention diluted across everything	LLM focuses on what matters
Scale	Works for 21 codes	Works for 500,000 codes

How RAG Works — The Key Insight

Embeddings: turn text into numbers that capture meaning.

```
"Water leaking from ceiling pipe burst"  →
[0.23, -0.14, 0.87, ...]
```

```
"PLUMB-001: Pipe Leak – Water leaking from
pipes..."  →  [0.21, -0.12, 0.85, ...]
```

These vectors are close in embedding space because the meaning is similar.
Search doesn't match keywords — it matches meaning.

The RAG Pipeline

1. LOAD

Read your documents

2. SPLIT

Break into chunks

3. EMBED

Convert to vectors

4. STORE

Save in vector store

5. RETRIEVE

Find k nearest vectors

6. GENERATE

LLM + context → answer

Indexing — happens ONCE

Every query

Is RAG Still Needed?

Claude handles ~200K tokens, Gemini 1M+. Can't we just stuff everything in?

Long Context Works For	RAG Wins For
• Single long documents • One-off analysis tasks • Simple architectures • When latency doesn't matter	• Large knowledge bases (500+ docs) • Repeated queries over same data • Cost-sensitive production systems • When you need speed at scale

For CBRE with 500 codes + work orders + manuals: RAG is the right call.

Why Not Just Chain Functions?

```
entities = extract(transcript)
codes = retrieve(entities)
classification = classify(entities, codes)
```

This works for a linear pipeline. But what happens when:

- ↺ Retrieval returns irrelevant results and you need to retry with a different query?
- 👤 The classifier's confidence is low and you want to route to a human?
- \+ You want to add a reflection step between extraction and retrieval?
- ⑂ Different transcript types need different paths through the pipeline?

You need conditional routing — and that's where simple function chaining breaks.

LangGraph: Graphs Over Glue Code

Model your pipeline as a graph — not a sequence of function calls.

Nodes

Python functions that do one thing — extract, retrieve, classify. Each node reads from state and writes back.

Edges

Connections between nodes. extract → retrieve → classify. The wiring of your pipeline.

State

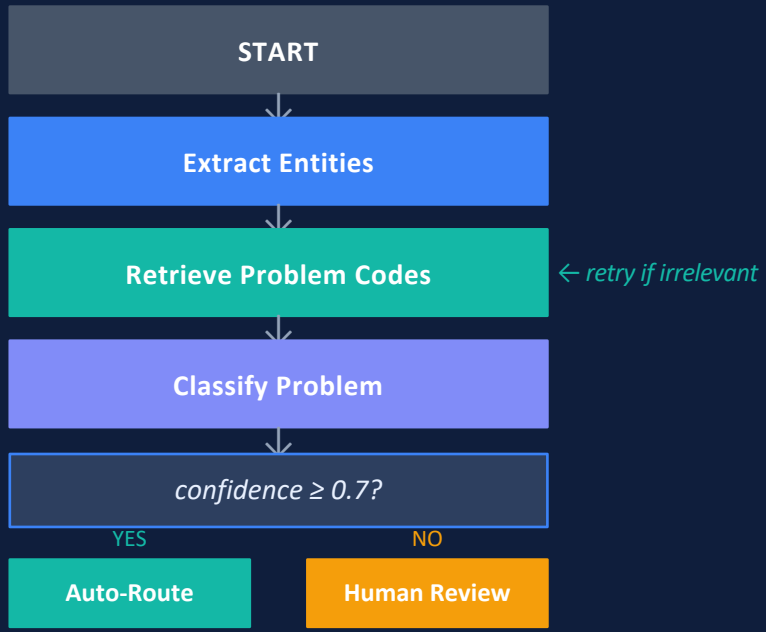
A shared dictionary that flows through the graph. Every node can read any previous node's output.

Conditional Edges

If confidence < 0.7, route to "human_review" instead of "auto_route." Decision logic is part of the graph structure.

The Mental Model + State

Think of it like a flowchart — with logic built into the arrows.



State: The Pipeline's Memory

```
class PipelineState(TypedDict):
    transcript: str
    entities: Optional[dict]
    retrieved_codes: Optional[list[dict]]
    classification: Optional[dict]
```

Every node reads from state and writes back to it:

`extract_entities`	reads transcript, writes entities
`retrieve_codes`	reads entities, writes retrieved_codes
`classify_problem`	reads entities + codes, writes classification

No globals. No 10-argument function calls. Just state.

Nodes + Wiring the Graph

Nodes are just Python functions. Wiring them takes 6 lines.

Nodes: Just Python Functions

```
def extract_entities(state):
    result = llm.with_structured_output(
        MaintenanceEntity).invoke(...)
    return {"entities": result.model_dump()}

def retrieve_codes(state):
    docs = retriever.invoke(query)
    return {"retrieved_codes": codes}

def classify_problem(state):
    result = llm.with_structured_output(
        Classification).invoke(...)
    return {"classification": result.model_dump()}
```

Each node returns only the keys it updates. LangGraph merges them into state.

Today: linear pipeline · Week 4: add conditional edges for HITL

Wiring the Graph

```
workflow = StateGraph(PipelineState)

workflow.add_node("extract", extract_entities)
workflow.add_node("retrieve", retrieve_codes)
workflow.add_node("classify", classify_problem)

workflow.add_edge(START, "extract")
workflow.add_edge("extract", "retrieve")
workflow.add_edge("retrieve", "classify")
workflow.add_edge("classify", END)

pipeline = workflow.compile()
```

Why This Matters vs. Plain Functions

Just Functions	LangGraph
Implicit flow	Explicit graph
Hard to branch	Conditional edges built-in
Manual state	Shared state object
Hard to visualize	draw_mermaid_png()

DEMO TIME

`langgraph_rag_demo.ipynb`

Walk through each node · Run a single transcript · Evaluate across all 10

Food for Thought

No right or wrong answers — open discussion prompts.

FT-1

At what scale does RAG become genuinely necessary vs. a nice-to-have? Is there a clear threshold, or is it more nuanced?

FT-2

The agentic RAG tutorial adds a "grade documents" step. Should every RAG system include a retrieval quality check, or is it overkill for some use cases?

FT-3

Our pipeline is linear: extract → retrieve → classify. Real calls are messy. What would a non-linear version look like? Where would you add branches or loops?

FT-4

Embeddings capture semantic similarity — but similarity ≠ relevance. Can you think of a case where retrieval would be misleading?

FT-5

The first CBRE automation was calling 911 for leaky faucets. How is what we're building fundamentally different? Or is it just a fancier version of the same idea?

Assignment — Part 1: Build Your Agentic RAG System

~3–4 hours total (setup, reading, and building)

Follow the official LangChain tutorial, adapted to CBRE:

```
Tutorial: Build a Custom RAG Agent with LangGraph → docs.langchain.com/oss/python/langgraph/agentic-rag
```

What to Adapt

× Blog posts from Lilian Weng	✓ **CBRE problem_codes.json (21 codes)**
× WebBaseLoader for fetching docs	✓ **json.load() for problem codes**
× Blog-specific retriever tool	✓ **Problem code retriever tool**
× Generic question answering	✓ **Maintenance call classification**

Milestones (in order)

- Index problem codes into a vector store and test retrieval
- Build generate_query_or_respond — LLM decides whether to retrieve
- Build grade_documents — checks if retrieved codes are relevant
- Build rewrite_question — improves query if retrieval was poor
- Build generate_answer — classifies using relevant codes
- Wire the full graph and test on 2-3 transcripts
- *Stretch: run on all 10 transcripts and compare accuracy to Week 2*

Part 2 + Reading List

Part 2: Think About It

Answer in a few sentences each:

1. When the LLM decides not to retrieve — was that the right call? Give an example.
2. Did the document grading step ever reject a relevant code or keep an irrelevant one?
3. Compare accuracy this week (extraction + RAG) vs. last week (extraction only). What changed?

Bring to Week 4

- Your notebook (or where you got stuck)
- Answers to the Think About It questions
- Ideas for human review checkpoints → feeds Week 4: HITL

Reading List

Is RAG Still Needed? — Alex Wang
The RAG vs. long-context debate. When is RAG the right architecture?

Build a Custom RAG Agent (LangGraph)
Your assignment reference — step-by-step tutorial to adapt.

LangGraph Documentation
Reference — nodes, edges, state, conditional routing.

LangChain RAG Conceptual Guide
Deep dive into RAG — embeddings, splitters, retrievers.

LangChain Docs Index
Master index for discovering all LangChain API pages.

QUIZ

Check Your Understanding

See the quiz document

Recap

Extraction → Classification	The pipeline now spans Extract → Retrieve → Classify → Route
RAG	Search first, pass only relevant context — scales to 500K+ codes
Embeddings	Vectors capture meaning, not just keywords — similarity search
Indexing vs. Querying	Index once, query every time — cost stays low at scale
LangGraph	Graphs over glue code — nodes, edges, state, conditional routing
State	Shared dict flows through the graph — no global vars or 10-arg functions
When to Use RAG	Large knowledge bases, repeated queries, production scale

NEXT WEEK

Week 4: HITL Systems

- Adding human review into the pipeline
- When should the AI decide alone?
- When should it ask for help?
- Conditional edges for HITL routing

Demo: langgraph_rag_demo.ipynb → Transcript → Extract → RAG Retrieve → Classify → Evaluate